EduAirControl: plataforma de microservicios con IoT para el monitoreo y análisis ambiental de espacios educativos

Jhon Jairo Cortez Vargas · María de los Ángeles Olaya García

Tecnólogo en Análisis y Desarrollo de Software · SENA Industria · Ficha 3145556

Instructora: Karol Daniela Correa · Neiva, Huila, Colombia · 10 de octubre de 2026 · Versión 1.0

Resumen

Las instituciones educativas rara vez cuentan con un lugar único donde consultar las condiciones ambientales de sus aulas: temperatura, humedad, calidad del aire y ruido suelen medirse de forma aislada o no medirse. Este artículo presenta EduAirControl, una plataforma web y móvil que centraliza esas mediciones, las asocia con el ambiente educativo y el sensor que las produjo, y permite consultarlas en tiempo real y analizarlas por día, semana, mes y año. El sistema se construyó con una arquitectura de microservicios en Java 17 y Spring Boot 4, un API Gateway con autenticación OAuth2/JWT, persistencia en PostgreSQL con Liquibase, mensajería con RabbitMQ, caché con Redis, un frontend en React y una aplicación móvil en Expo/React Native. Un prototipo con ESP32, DHT11, KY-038 y MQ-135 valida la captura física de datos. El proyecto incluye siete módulos de backend con 285 métodos de prueba automatizada y un plan de pruebas de ocho niveles. Se documentan también las limitaciones: el firmware definitivo que envía lecturas a la API y la calibración del MQ-135 quedan pendientes, y la ejecución completa del plan de pruebas aún no se ha cerrado.

Palabras clave: monitoreo ambiental, Internet de las cosas, microservicios, Spring Boot, ESP32, espacios educativos, arquitectura hexagonal.

1. Introducción

Las condiciones ambientales de un aula influyen en el confort y la atención de quienes la usan. Aun así, la información que producen los sensores suele quedar dispersa: cada dispositivo muestra su valor, nadie conserva el historial y no hay una relación clara entre una medición y el espacio donde se tomó. Quien administra los espacios no puede comparar cómo cambian las condiciones con el tiempo ni decidir con datos.

EduAirControl nace para cerrar esa brecha. Su propuesta de valor tiene cuatro ejes definidos en la visión del producto: **centralización** de la información en una sola plataforma, **contexto** (cada medición ligada a un ambiente, un sensor y una hora), **comprensión** mediante paneles y análisis históricos, y **protección** del acceso con registro, autenticación, roles y auditoría. El producto no promete control físico automático ni gestión predictiva; esas capacidades quedan fuera del alcance actual.

2. Planteamiento del problema

Los usuarios y administradores de una institución necesitan entender el estado ambiental de sus espacios, pero la información de los sensores no está centralizada ni analizada. De ahí surgen cinco dimensiones del problema:

Dimensión	Problema	Implicación para el producto
Acceso	No hay un lugar claro para consultar la información	Panel autenticado
Centralización	Las mediciones quedan aisladas del espacio	Gestión de ambientes, sensores y mediciones
Comprensión	Los valores crudos no explican la evolución	Análisis por día, semana, mes y año
Trazabilidad	Una medición solo vale si se conoce su origen y hora	Relación sensor–ambiente–marca de tiempo
Gobierno	Debe controlarse quién ve y administra	Roles, permisos, sesiones y auditoría

3. Objetivos

Objetivo general. Desarrollar una plataforma que permita monitorear, gestionar y analizar las condiciones ambientales de espacios educativos mediante datos de sensores IoT.

Objetivos específicos.

- Implementar autenticación y autorización con roles (SUPER_ADMIN, ADMIN, USER y VIEWER).
- Gestionar campus, ambientes educativos, sensores y dispositivos.
- Recibir, validar, almacenar y analizar mediciones, y generar alertas a partir de umbrales.
- Ofrecer interfaces web y móvil con panel, favoritos, preferencias, idiomas y accesibilidad.
- Validar la captura física de datos con un prototipo basado en ESP32.
- Documentar y probar el sistema de forma que sus resultados sean verificables.

4. Metodología

El equipo trabajó con desarrollo guiado por especificación (SDD): primero se documentó el contexto, el dominio, el producto y los requisitos, y después la arquitectura, los datos y los contratos de API. La documentación se organizó en carpetas de gobierno, contexto, dominio, producto, requisitos, arquitectura, datos, API, UML, microservicios, DevOps, calidad, operación y control del proyecto.

Las decisiones de diseño se registraron como ADR (registros de decisión de arquitectura). Entre ellas, cada servicio es un proyecto independiente con su propia base lógica (un esquema dedicado por servicio) y sigue una **arquitectura hexagonal**, que separa el dominio de los adaptadores de entrada y salida. Los contratos de los servicios se describen en OpenAPI. La calidad se apoyó en pruebas automatizadas, un smoke test de extremo a extremo y un pipeline de integración continua; la contenerización se hizo con Docker Compose y se dejó preparada la ruta hacia Kubernetes.

5. Arquitectura y solución implementada

5.1 Visión general

El sistema sigue un modelo cliente–servidor. La aplicación web y la móvil consumen una API HTTP/JSON expuesta por un **API Gateway** (Spring Cloud Gateway, puerto 8080) que enruta, valida el JWT, aplica límite de solicitudes y propaga un identificador de correlación. Detrás del gateway trabajan seis microservicios:

Servicio	Responsabilidad	Puerto
ms-security	Registro, inicio y cierre de sesión, OAuth2/OIDC, JWT RS256, roles e instituciones	8081
ms-classroom-management	Campus, tipos de ambiente y ambientes educativos	3002
ms-environment-monitoring	Mediciones, panel, análisis, umbrales y alertas	3003
ms-sensor-management	Catálogo, sensores, instalaciones, variables y dispositivos	3004
ms-user-experience	Favoritos, preferencias y calificaciones	3006
ms-user-management	Perfil de usuario	3007

5.2 Pila tecnológica

Capa	Tecnología
Backend	Java 17, Spring Boot 4.0.5, Spring Data JPA
Seguridad	Spring Authorization Server, JWT firmado con RS256, inicio de sesión con Google y Facebook (flujo de código de autorización)
Datos	PostgreSQL 15, Liquibase (`ddl-auto=validate`)
Mensajería y caché	RabbitMQ 3.13, Redis 7 (lista de revocación de tokens)
Web	React 19, Vite 8, React Router, i18next, Recharts
Móvil	Expo 54, React Native 0.81, AsyncStorage, BLE PLX
Contenedores y CI	Docker Compose, pipeline de integración continua, manifiestos Kubernetes

5.3 Flujo de información ambiental

1. Los sensores capturan las variables del espacio educativo.
2. El ESP32 procesa las lecturas y, en la integración final, las envía al punto de ingesta.
3. El backend valida la clave del dispositivo (`X-API-Key`), procesa la medición y la guarda en PostgreSQL; los eventos viajan por RabbitMQ.
4. La web y la app móvil consultan la API y presentan mediciones, estados, análisis y alertas.

5.4 Seguridad

El acceso de usuarios se basa en JWT con expiración y revocación (al cerrar sesión el token se registra en Redis y su reutilización se rechaza con 401). Los roles separan las operaciones de administración de las de consulta. Los dispositivos usan una clave `X-API-Key` que se muestra una sola vez al crearse o rotarse. Los secretos se configuran por variables de entorno y CORS es configurable.

5.5 Prototipo IoT

Componente	Función en el prototipo
ESP32	Controlador y comunicación Wi-Fi; aprovisionamiento por BLE desde la app móvil
DHT11	Temperatura y humedad del aire
KY-038	Nivel relativo de sonido (salida analógica)
MQ-135	Variación relativa de gases y calidad del aire

El MQ-135 requiere calibración y adaptación de voltaje antes del conversor analógico del ESP32; por eso **no se presenta como medidor calibrado de CO_2 en ppm**. Los archivos entregados

contienen pruebas de DHT11 y KY-038 a 115 200 baudios.

6. Calidad y pruebas

El plan de pruebas (versión 1.0) define ocho niveles: unitarias, integración, contrato, funcionales, sistema, IoT, regresión y no funcionales. Incluye una matriz de 60 casos funcionales (seguridad, ambientes, sensores, monitoreo, web, móvil e IoT), criterios de entrada y salida, clasificación de resultados (PASS, FAIL, BLOCKED, NOT RUN, NOT APPLICABLE) y gestión de defectos.

El inventario automatizado del código fuente muestra la siguiente distribución:

Servicio	Métodos de prueba
ms-environment-monitoring	83
ms-security	57
ms-sensor-management	55
ms-classroom-management	33
API Gateway	27
ms-user-management	20
ms-user-experience	10
Total	**285**

Estas pruebas cubren servicios de aplicación, controladores, JWT, lista de revocación en Redis, limitación de solicitudes, consumidores de eventos y reglas del dominio de alertas y análisis.

Estado de la ejecución. El registro de ejecución del 9 de octubre de 2026 se encuentra en estado de borrador: todos los casos están en NOT RUN, salvo IOT-008 (BLOCKED por falta del firmware de envío). Como antecedente, un reporte del 7 de octubre muestra 10 de 10 pruebas aprobadas en ms-user-experience y, en ms-environment-monitoring, 28 aprobadas y 58 con error. Ese error se debe a un bean duplicado causado por una clase compilada obsoleta en `target/`, que probablemente se resuelve con una compilación limpia; si persiste, se tratará como defecto. Estos datos no se presentan como resultado final de la ronda de pruebas.

7. Resultados

- Se obtuvo una plataforma con siete módulos de backend (gateway y seis microservicios), una aplicación web y una aplicación móvil.
- Se cubren las ocho capacidades del alcance mínimo (autenticación, ambientes, sensores, monitoreo, panel, análisis histórico, favoritos y calificación de ambientes).
- Se entregó un conjunto documental completo: especificación de requisitos, análisis, propuesta técnica, diseño, base de datos, plan de pruebas, calidad, implementación y manuales técnico, de instalación, de administración y de usuario.
- El prototipo físico demuestra la lectura de temperatura, humedad y sonido, y el aprovisionamiento por BLE.

Limitaciones. (1) No existe aún el firmware definitivo que envíe mediciones a la API. (2) El MQ-135 no está calibrado. (3) No hay pruebas automatizadas del frontend. (4) No se realizaron pruebas de carga, de penetración certificada ni de recuperación ante desastres. (5) El plan de pruebas aún debe ejecutarse por completo y revisarse por los responsables técnico y funcional.

8. Conclusiones

EduAirControl demuestra que es viable centralizar datos ambientales de espacios educativos en una plataforma segura, trazable y escalable, con una arquitectura de microservicios hexagonales y un prototipo IoT de bajo costo. La separación por servicios, los contratos OpenAPI y la gestión de cambios con Liquibase facilitan evolucionar el sistema sin romper lo existente.

La principal lección del proceso es que documentar y probar con honestidad vale tanto como construir: distinguir lo ejecutado de lo pendiente permite que los resultados sean verificables y que las limitaciones sean decisiones conscientes, no sorpresas.

Trabajo futuro. Completar el firmware de ingesta con reconexión Wi-Fi; calibrar el MQ-135 o sustituirlo por un sensor de CO_2 certificado; ejecutar y cerrar el plan de pruebas; incorporar pruebas automatizadas de web y móvil; realizar pruebas de carga y seguridad; y desplegar en un ambiente de staging para validar el rollback.

Referencias

1 EduAirControl. *Manual técnico*, versión 1.1. SENA Industria, 2026.
2 EduAirControl. *Manual de instalación*. SENA Industria, 2026.
3 EduAirControl. *Plan de pruebas*, versión 1.0, 9 de octubre de 2026.
4 EduAirControl. *Documentación de producto, arquitectura y requisitos* (carpetas 01 a 15 del repositorio de documentación), 2026.
5 Cockburn, A. *Hexagonal Architecture* (Ports and Adapters), 2005.
6 VMware/Broadcom. *Spring Boot*, *Spring Cloud Gateway* y *Spring Authorization Server*: documentación oficial.
7 PostgreSQL Global Development Group. *PostgreSQL 15 Documentation*.
8 Espressif Systems. *ESP32 Series Datasheet*.